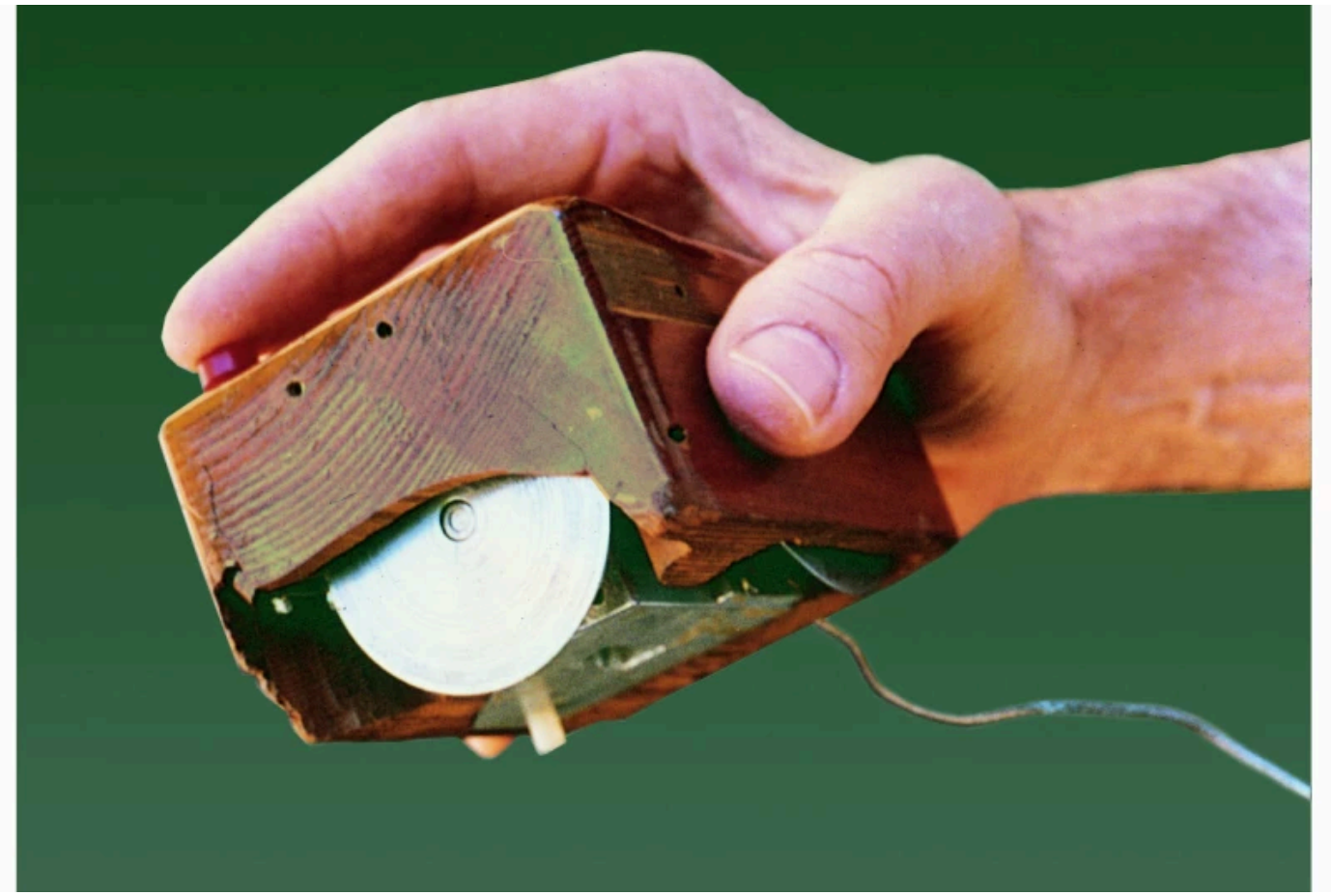
Introduction

Decades before personal computers, the concept of a pointing device to facilitate human-computer interaction emerged from research. The first prototype of such a device was developed in the early 1960s by Douglas Engelbart and William English at the Stanford Research Institute. The first demo of the so called “X-Y Position Indicator for a Display System”, a device with 2 metal wheels to control x and y movement, took place in 1968 during what is now known as **“The Mother of All Demos”**, where the researchers presented it along the first instances of hypertext, real-time collaboration, and windowed interfaces. The term “mouse” was coined by their colleagues due to the device’s resemblance to a small rodent with a tail (the cable).



The first computer mouse was made of wood and featured two wheels to control X and Y movement (Image: SRI International)

The first commercially available mouse was the German Telefunken Rollkugel, released in 1968 as a peripheral for the SIG 100 terminal and Telefunken TR 440 computer system. It already featured the track ball mechanism popular until the early 2000s. Later, in 1973, and with the help of Bill English, Xerox PARC developed the Alto personal computer, which featured a three-button ball mouse as part of its innovative graphical user interface. None of these devices were widely adopted until the 1980s and the release of the Apple Lisa and Macintosh computers, and later the IBM PS/2 systems, which popularized the mouse as a standard input device.

Mouse input grew to be the dominant form of interaction with digital systems, before the rise of touchscreens and mobile devices, marking an important paradigm shift in human-computer interaction. The mouse, together with graphical user interfaces (GUIs), enabled more intuitive and efficient interactions, making computers accessible to a broader audience beyond technical experts. Even today, the mouse remains a fundamental tool for navigating and interacting with digital environments, especially in desktop computing contexts.

That' s why, in this lab, you will learn how to configure and read input from the PC Mouse. To do so we will learn about its low-level communication interface with the PC, and how mouse movements and button presses are reported to the operating system, before being passed to windowing systems.

Plan

This will be a one class lab where you are expected to understand the communication mechanisms between the PC and the PC Mouse and how to interact with the PC Mouse controller. We'll start by learning about the `i8042` auxiliary device features, and then you will develop two functions that read and display mouse movements by processing mouse data packets upon receiving `i8042` mouse interrupts.

We advise you to read everything carefully, refrain from a blind use of AI, and use the support of the teaching assistants whenever you feel stuck. Good luck and have fun!

References

Here you have some relevant files and documentation for the PC Mouse lab:

- lab4.c file that contains the test function stubs that you will develop
- Doxygen documentation
- 8042 functional description from the IBM Technical Reference Manual
- Synaptics TouchPad Interfacing Guide
- Andries Brouwer's The PS/2 Mouse, Ch. 13 of Keyboard scancodes
- Adam Chapweske's The PS/2 Mouse Interface (Similar to the previous reference, as far as this lab is concerned.)
- Data sheet of an 8042-compatible IC

The `i8042` Mouse Controller

As you may have noticed, the PC keyboard and mouse share the same `i8042` controller, which is responsible for managing communication between these I/O devices and the processor. The `i8042` was integrated for the first time in the IBM PC/AT to handle keyboard operation. As keyboards evolved, they introduced new features such as LED indicators for Caps Lock and Num Lock, as well as multiple scan code sets, which required bidirectional communication between the host and the keyboard. Older systems could not support this, as they only provided unidirectional input, from the keyboard to the host, which was effectively a passive listener of keyboard events. The `i8042` controller addressed this limitation by providing programmable, bidirectional communication, using the same mechanisms to communicate with the keyboard that Lab 3 alludes to.

Later, with the introduction of the PS/2 mouse in 1987 alongside the IBM PS/2 systems, the `i8042` was updated to support mouse input as well, referred to as the `auxiliary device` in its documentation. This update added a second input port, extended the controller firmware with new commands to manage the mouse device, introduced a dedicated interrupt line (**IRQ 12**), and standardized keyboard and mouse input protocols.

> **Note**
>
> The `i8042` controller is also known as the **Keyboard Controller (KBC)**, as it was initially designed to manage only keyboard input.

Interacting with the Controller at the Mouse

Similarly to the keyboard, the mouse contains a configurable microcontroller that communicates with the `i8042` controller using the PS/2 protocol. The `i8042` acts as an intermediary between the mouse and the CPU.

When the `i8042` controller is configured to multiplex mouse data alongside keyboard input, it functions in PS/2 mode. In this mode, the KBC supports commands specifically aimed at controlling the mouse, such as enabling or disabling the mouse interface, or enabling/disabling interrupts when a byte is received from the mouse. KBC commands are written to port `0x64`, and their arguments are written to port `0x60`.

To send a command directly to the mouse, the driver first writes the KBC command `0xD4` (Write to Auxiliary Device) to port `0x64`, and then writes the actual mouse command byte to port `0x60`. After each byte is sent, the mouse responds with an acknowledgment byte, which the KBC places in its output buffer; the driver must read this byte from port `0x60`. If a mouse command requires additional argument bytes, the same process is repeated for

each argument: write `0xD4` to port `0x64`, the argument byte to port `0x60`, and read the acknowledgment from the KBC.

Once the final byte of the command (or its last argument) is received and acknowledged, the mouse executes the command. If the command produces a response, the mouse sends it to the KBC, which places it in the output buffer for the driver to read from port `0x60`.

Like with the keyboard, the communication bewteen mouse and KBC is not immediate, and thus you should not expect to receive the acknowledgment byte, or the response the mouse sends to a command, immediately after writing them. As suggested for Lab 3, rather than waiting indefinitely, or until the KBC reports a time-out, your code should give the KBC and the mouse enough-time to respond, retry a few times on time-out, and finally give up. Given that the time intervals to consider are in the order of tens of ms, once again, it is not appropriate to use `sleep()`. Use the functions `tickdelay` and `micros_to_ticks`, presented in Lab 3.

Handling Command Errors

As described above, when you send a byte to the mouse, the mouse will respond with an acknowledgement byte. If it responds with `0xFE`, which signals an error, you need to resend the entire command from the beginning, not only the last byte.

From the Synaptics Interfacing Guide, page 31:

When the host (driver) gets an `0xFE` response, it should retry the offending command. If an argument byte elicits an `0xFE` response, the host should retransmit the entire command, not just the argument byte.

> ⚠ **Warning**
>
> The mouse may respond with `0xFC` (Error) in certain situations, such as when it receives an argument after an `0xFE` response. In such cases, the driver should also attempt to resend the command from the beginning.

> 💡 **Tip**
>
> Conversely, the acknowledgment byte `0xFA` indicates that the mouse has successfully received the byte and is ready for the next one. This byte does not imply that the command has been fully executed, only that the byte was received correctly. A complete response from the mouse, if applicable, may follow an initial `0xFA` acknowledgment with additional bytes.

Order	Description	Action to do	Notes

1	Request forwarding of byte (command) to the mouse	Write `0xD4` to port `0x64`	
2	Write the byte (command)	Write the code for the command to port `0x60`	See Synaptics Interfacing Guide, page 33, for the command list
3	Read the acknowledgement byte received from the mouse	Read the acknowledgement byte from port `0x60`	In case of success, acknowledgement byte is `0xFA` . In case of error the value will differ, see above
4	If command has arguments	Repeat steps above for each byte of the argument(s); including reading the acknowledgement byte	Again, there might be an error, see above
mouse executes command			
5	If command has response	read response from port `0x60`	

Table 1: Instructions on how to send commands to the mouse.

> **Important**
>
> Don't forget to check the KBC status register before writing to ports `0x60` or `0x64` , to ensure that the input buffer is not full and that it is safe to write. Also, when reading from port `0x60` , check that the output buffer is full and that there are no parity or timeout errors.

Mouse Modes

As described in Adam Chapweske's The PS/2 Mouse Interface, the mouse can operate in different modes.

The two primary modes are:

Stream Mode

In this mode, the mouse continuously sends data packets to the host whenever there is movement or button activity. This is the default mode of operation. It is often paired with the `i8042` interrupt mechanism to notify the host of incoming data. When in this mode, we can control whether the mouse should send data packets automatically by enabling or disabling data reporting. To enable data reporting, send the command `0xF4` to the mouse. To disable data reporting, send the command `0xF5` .

Remote Mode

In this mode, the mouse only sends data packets when polled by the host. This mode is less common and is typically used in specific applications where continuous data is not required.

> **Important**
>
> Before sending any command to the mouse, you must disable data reporting to prevent interference from packets sent by the mouse.
>
> From the Synaptics Interfacing Guide, page 33:
>
> If the device is in Stream mode (the default) and has been enabled with an Enable (`0xF4`) command, then the host should disable the device with a Disable (`0xF5`) command before sending any other command.

Mouse Data Packets

Just as the keyboard sends scancodes to indicate key presses and releases, the mouse sends data packets to report movements and button states. The packets sent by a mouse to its controller are composed by several bytes. Different mouse types use different packet types according to their features. For example, whereas the PS/2 mouse uses a 3-byte packet, the Microsoft Intellimouse, which includes a scrolling-wheel, uses a 4-byte packet mouse. This simple interface between the mouse and the KBC allows for the support a wide range of mice, from very simple to very sophisticated. Andries Brouwer's The PS/2 Mouse, Ch. 13 of Keyboard scancodes details the format of the standard PS/2 mouse packet and the extended packet formats used by some other more advanced mice.

> **Note**
>
> In this Lab, you'll only use the standard PS/2 protocol, presented in the lectures.

The standard PS/2 mouse packet consists of 3 bytes, with the following format:

	Bit 7	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Byte 1	Y overflow	X overflow	Y sign bit	X sign bit	Always 1	Middle Btn	Right Btn	Left Btn
Byte 2	X movement							
Byte 3	Y movement							

Table 2: Standard PS/2 mouse packet format.

Apart from the button states and movement data, the first byte contains overflow and sign bits for both X and Y movements. The overflow bits indicate whether the movement in that direction exceeded the maximum value that can be represented in a single byte (i.e., greater than 255 or less than -256). The sign bits indicate whether the movement is positive or negative. Then the second and third bytes represent the actual movement in the X and Y directions, respectively, using two's complement representation for negative values. This means that if the sign bit is set (1), the corresponding movement byte should be converted to the corresponding negative value.

Synchronizing with the Mouse

The device driver must be synchronized with the mouse to ensure that when it processes a mouse data packet, the 3 bytes it uses all belong to the same packet. For example, using two bytes of a packet and the first byte of the following packet may lead to incorrect behavior.

The challenge is that the bytes of the PS/2 packet do not carry an identification or sequence, and therefore it is not easy to detect that the driver is not in sync with the mouse. A naïve solution relies on the fact that bit 3 of byte 1 is always 1. Thus, if the byte the driver is expecting is the first one, and bit 3 of the byte received is 0, the byte received cannot be the first one and the driver is not in sync with the mouse.

Although this does not guarantee that your "driver" will always be in sync, as other bytes can have bit 3 set to 1, this can be used to eventually synchronize the driver with the mouse. Actually, the efficacy of this approach will depend on the mouse usage pattern: if the user starts only by clicking the mouse, then bit 3 of all packets but the first will be 0, and the "driver" will get in sync with the mouse after 1 or 2 bytes.

The Mouse in Modern Systems

In modern systems, the discrete `i8042` chip has been integrated into the motherboard's Super I/O controller or is emulated by the BIOS/UEFI to maintain legacy support for older operating systems. However, the software-facing interface remains identical to the original design, communicating with the CPU through I/O ports 0x60 and 0x64, both for keyboard and mouse data.

Nowawdays, USB has largely supplanted PS/2 as the primary interface for keyboards and mice, offering advantages such as hot-plugging (i.e., the ability to connect and disconnect devices without restarting the computer), higher data transfer rates, and support for a broader range of features.

> **Note**
>
> Did you know that USB devices are polled rather than using interrupts like PS/2 devices?

> While the PS/2 protocol utilizes an asynchronous interrupt-driven model, USB HID (Human Interface Device) peripherals operate on a synchronous polling model. In this paradigm, the host controller periodically queries the device for state changes at a predefined sampling rate (typically 125 Hz to 1000 Hz). Consequently, USB communication is governed by the host's schedule rather than the peripheral's activity.

Nevertheless, many systems still include PS/2 ports for compatibility with legacy hardware, and for the reliable, low-latency input that PS/2 devices can provide, especially in embedded systems and industrial applications, or in scenarios where USB support is not available or desired.

PS/2 is still a very much alive technology that stands the test of time!

Reading mouse movements

In this Lab, you will be implementing two functions that read and display mouse movements and process mouse data packets using the `i8042` mouse interrupts.

Stopping after some packets - `int mouse_test_packet(uint32_t cnt)`

The first function of this lab will read and display in a friendly way a given number of PS/2 mouse packets from the KBC, given as the argument `cnt`. To do so, you will need to configure the mouse properly, subscribe mouse interrupts, implement an appropriate Interrupt Handler (IH), and parse and display the mouse packets. In the end, you must reset the mouse to Minix's default configuration.

Minix, despite starting in terminal mode, configures the mouse to operate in stream mode and registers a mouse interrupt handler by default. However, Minix does not enable data reporting by the mouse, so the mouse does not send any data packets until data reporting is enabled, not generating any interrupts. To help you get started, we provide you with the function `int mouse_enable_data_reporting()` that enables data reporting, documented in doxygen here.

> **Important**
>
> Remember that mouse interrupts must be subscribed in exclusive mode (use IRQ_REENABLE and IRQ_EXCLUSIVE on IRQ 12), otherwise, Minix's default mouse interrupt handler may interfere with your implementation.

> **Important**
>
> The `i8042` generates an interrupt on every byte received. Therefore, your IH must only read one byte from the output buffer each time it is invoked.

Your IH must be implemented in a function with the following definition, remembering the braces around the function name to allow proper parsing by the LCF:

```
#include <lcom/lcf.h>

void (mouse_ih)(){
    // Your code here
}
```

Since the IH takes no arguments and returns no values, the passing of data between `mouse_ih` and the other functions of your implementation must be done via global variables, preferrably static, i.e, the variable should not be visible outside the file, and, therefore, not imported as an external variable.

When you receive the last byte of a mouse packet, the 3rd, your program should parse it and print it on the console, by calling the following function, provided you as part of the LCF (doxygen documentation here):

```
#include <lcom/lcf.h>

void mouse_print_packet(struct packet *pp);
```

This function accepts as an argument a pointer to a `struct packet`, documented in doxygen here:

Finnaly, after reading and displaying the required number of packets, your function must disable data reporting. You will need to implement this functionality yourself, by writing the appropriate command to the mouse. You can find more information about how to do this in the references provided in the Introduction page and in the The `i8042` Mouse Controller page.

> **Important**
>
> Remember how build an appropriate Makefile to compile your code with the LCF, as explained in the Development Environment and the LCOM Framework page. Include the necessary source files, header files and libraries, as well as the target for builing the final mouse library.

Testing with the LCF

To test you implementation of this function, you must run your program in test mode, by executing the following command in the terminal of your Minix VM:

```
minix$ lcom_run lab4 "packet <no. of packets> -t <test no.>"
```

Where `<no. of packets>` is the number of packets to be read and displayed, and `<test no.>` is the number of the test to be executed by the LCF.

In test mode, the LCF will check if your program behaves as expected; in other words, if the output are provided when and as expected. For the mouse tests, the LCF injects mouse packets to your program, thus emulating the mouse and its input. Unlike a real user, the LCF injects packets in quick succession, resulting in a faster execution of your program than when using a real mouse.

Before exiting, the LCF prints a message with the result of the test, and the sequence of packet bytes injected. The value of these bytes are represented in hexadecimal, but they are not prefixed with `0x`. We hope that this will help you better debug your code, if it fails a test.

For test numbers 1 to 5, each test function injects the sequence of packets every time you run the test with the same arguments. Each test number corresponds to a different sequence of packets, aimed at testing different aspects of the mouse functionality, as described below.

1. The packets report only mouse buttons' events
2. Packet sequence corresponding to horizontal movemnet
3. Packet sequence corresponding to vertical movement
4. Packet sequence corresponding to mouse movement from left to right with a positive slope
5. In this case, the packets report various events, including mouse buttons being pressed/released and both upward and downward mouse movements.

For test number 0, the sequence of packets injected is random. We suggest that you use this test number only after running successfully test numbers 1 to 5. Running test number 0 repeated times, will allow you to more extensively test your code.

> **Tip**
>
> To manually test your code, ensure the mouse input is captured by the VM by clicking inside the VM window.

> **Note**
>
> Remember that in test mode, the trace and output log functionalities of the LCF are disabled. Nevertheless, you can still use `printf` and check its output in the `/var/log/messages` file.

Stopping after idle time - `int mouse_test_async(uint8_t idle_time)`

This function should do essentially the same as `mouse_test_packet`, i.e. it should display the packets received from the mouse. However, it has a different exit condition. `mouse_test_async` should terminate if it receives no packets from the mouse for the number of seconds specified in its argument `idle_time`. For measuring the time you must use `timer 0`'s interrupts.

> **Note**

> You must not change the configuration of `timer 0`. Optionally, you can determine `timer 0` interrupt frequency with the help of the kernel call `sys_hz`, ensuring that has not been tampered with.

Like in `mouse_test_packet`, `mouse_test_async` must enable data reporting and subscribe mouse interrupts. Also like `mouse_test_packet`, your function must restore the original state, unsubscribing mouse interrupts and disabling data reporting before returning. Additionally, `mouse_test_async` must also subscribe timer interrupts and unsubscribe them before returning.

Unlike `mouse_test_packet`, you must implement the functionality of `mouse_enable_data_reporting`, using the `0xD4` command, similarly to how you will implement the functionality to disable data reporting.

Testing with the LCF

To test you implementation of this function, you must run your program in test mode, by executing the following command in the terminal of your Minix VM:

```
minix$ lcom_run lab4 "async <idle_time (seconds)> -t <test no.>"
```

Where `<idle_time>` is the number of seconds of inactivity after which the function should terminate, and `<test no.>` is the number of the test to be executed by the LCF.

The tests for this function are similar to those of `mouse_test_packet`, with the difference that the LCF will wait for the specified idle time before terminating your program.